

Report 1: Inter-Process Communication Architecture

uORB Messaging, XRCE-DDS Bridge, AutosoarControl, and AutosoarStatus

PX4 Autosoar Implementation — Defence Technical Report

Radhouene — Fixed-Wing Autosoar Project

April 2026

Contents

1	Introduction	2
2	Original System: uORB and the Absence of Soaring IPC	2
2.1	uORB Publish-Subscribe Middleware	2
2.2	XRCE-DDS Bridge	2
2.3	Original dds_topics.yaml (no soaring)	3
3	Modified System: AutosoarControl Message Design	3
3.1	Design Rationale: From Two Booleans to One Integer	3
3.2	Complete AutosoarControl Message	3
3.3	Field Usage Matrix	4
3.4	Registration in dds_topics.yaml	4
3.5	AutosoarStatus message (FMU → companion)	5
3.6	Staleness Watchdog	5
3.7	CLI Override Path	5
4	Summary of Changes	6

1 Introduction

This report describes the inter-process communication (IPC) infrastructure used to command the autosoar system from a ROS 2 companion computer to the PX4 flight management unit (FMU). It covers:

- the uORB publish-subscribe middleware;
- the XRCE-DDS bridge that connects ROS 2 to uORB;
- the original messaging approach (boolean flags);
- the redesigned `AutosoarControl` message using a single enumerated mode integer;
- the `AutosoarStatus` message published by the FMU so the companion can observe `soar:off` (including the 5 s DDS inhibit deadline) and staleness-forced shutdowns.

2 Original System: uORB and the Absence of Soaring IPC

2.1 uORB Publish-Subscribe Middleware

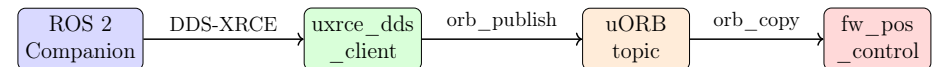
PX4 uses *uORB* (micro Object Request Broker) as its internal IPC bus. It is a strongly-typed, zero-copy, publish-subscribe system running entirely inside the NuttX RTOS process space. Each *topic* is defined by a `.msg` file and compiled into a C struct. Publishers call `orb_publish()`, subscribers call `orb_subscribe()` and `orb_copy()`.

Key properties:

- **Single writer, multiple readers:** uORB supports one active publisher per topic instance. Multiple modules may subscribe.
- **Non-blocking:** `orb_check()` tests for new data without blocking. Blocking waits use `px4_poll()`.
- **Fixed-size structs:** every message has a 64-bit `timestamp` field plus typed payload fields.

2.2 XRCE-DDS Bridge

The `uxrce_dds_client` module implements the *DDS-XRCE* (eXtremely Resource Constrained Environments) protocol, bridging uORB topics to a ROS 2 DDS domain:



The bridge is configured in `dds_topics.yaml`. Topics listed under `subscriptions` flow from ROS 2 *into* the FMU (`/fmu/in/`); topics under `publications` flow out.

2.3 Original dds_topics.yaml (no soaring)

In the original *Music* codebase, `dds_topics.yaml` contained no entry for autosoaring. The companion computer had no standardised channel to command soaring behaviour; workarounds relied on MAVLink parameter setting or ad-hoc MAVLink commands.

3 Modified System: AutosoaringControl Message Design

3.1 Design Rationale: From Two Booleans to One Integer

The first prototype used two boolean fields: `glide_mode_enabled` and `thermal_mode_enabled`. This creates $2^2 = 4$ possible combinations, two of which are ambiguous:

glide	thermal	Interpretation
false	false	powered flight
true	false	glide mode
false	true	thermal mode
true	true	ambiguous — both asserted

Furthermore, a separate PX4 parameter `FW_GLIDE_MODE` (integer 0 or 1) controlled whether the glide airspeed used a fixed value or the polar-optimal speed, creating a *hidden third dimension* invisible in the uORB message.

The redesign encodes all five states in a single `uint8 soaring_mode` field:

Value	Constant	TECS behaviour	Navigator state
0	SOARING_OFF	powered	unchanged
1	SOARING_GLIDE_FIXED	gliding, fixed EAS	AUTO_MISSION
2	SOARING_GLIDE_POLAR	gliding, polar EAS	AUTO_MISSION
3	SOARING_THERMAL_LOITER	gliding	AUTO_LOITER (navigator-guided)
4	SOARING_THERMAL_BANK	gliding	AUTO_LOITER (roll override)

3.2 Complete AutosoaringControl Message

Listing 1: AutosoaringControl.msg (new)

```
uint64  timestamp           # microseconds since boot

uint8   soaring_mode       # see constants below
uint8   SOARING_OFF        = 0
uint8   SOARING_GLIDE_FIXED = 1
uint8   SOARING_GLIDE_POLAR = 2
uint8   SOARING_THERMAL_LOITER = 3
uint8   SOARING_THERMAL_BANK = 4

# Loiter geometry (mode 3 only)
float32 loiter_radius_m     # [m]; NaN => bank-derived minimum
```

```
float32 loiter_lat          # [deg]; NaN/0 => current position
float32 loiter_lon          # [deg]; NaN/0 => current position

# Optimisation commands (NaN => PX4 parameter default)
float32 airspeed_cmd        # [m/s EAS] mode 1 only
float32 bank_angle_cmd      # [deg]      modes 3 and 4
bool    loiter_clockwise    # true=CW, false=CCW

# Metadata
uint8   soaring_phase
uint8   SOARING_PHASE_CRUISE = 0
uint8   SOARING_PHASE_GLIDING = 1
uint8   SOARING_PHASE_SEARCHING = 2
uint8   SOARING_PHASE_CENTERING = 3
uint8   SOARING_PHASE_THERMAL = 4
```

3.3 Field Usage Matrix

Field	Mode 0	Mode 1	Mode 2	Mode 3	Mode 4
airspeed_cmd	—	✓	—	—	—
loiter_radius_m	—	—	—	✓	—
loiter_lat/lon	—	—	—	✓	—
bank_angle_cmd	—	—	—	✓	✓
loiter_clockwise	—	—	—	✓	✓
soaring_phase	✓	✓	✓	✓	✓

3.4 Registration in dds_topics.yaml

Companion → **FMU** (subscription side of the client):

Listing 2: Addition to `dds_topics.yaml` (subscriptions)

```
subscriptions:
...
- topic: /fmu/in/autosoaring_control
  type: px4_msgs::msg::AutosoaringControl
```

This enables the bridge to forward ROS 2 messages published on `/fmu/in/autosoaring_control` into the FMU uORB bus.

FMU → **companion** (publication side):

Listing 3: Addition to `dds_topics.yaml` (publications)

```
publications:
...
- topic: /fmu/out/autosoaring_status
  type: px4_msgs::msg::AutosoaringStatus
```

The `fw_pos_control` module publishes `autosoaring_status` on uORB; the bridge exposes it to ROS 2 as `/fmu/out/autosoaring_status`.

3.5 AutosoaringStatus message (FMU → companion)

Listing 4: AutosoaringStatus.msg

```
uint64 timestamp
uint8 source
uint8 SOURCE_CLI_SOARING_OFF      = 0    # operator soar:off (
    CUSTOM_0, mode 0)
uint8 SOURCE_STALENESS_WATCHDOG   = 1    # companion silent >2 s
uint8 SOURCE_PERIODIC             = 2    # 2 Hz heartbeat while
    soaring active
uint8 soaring_mode_effective      # same encoding as
    AutosoaringControl.soaring_mode
uint64 dds_inhibit_until_us       # absolute HRT time [us]; 0 = no
    DDS inhibit
uint8 fmu_soaring_phase
uint8 SOARING_PHASE_CRUISE        = 0    # powered cruise (soaring
    OFF)
uint8 SOARING_PHASE_GLIDING       = 1    # engine-off glide along
    mission
uint8 SOARING_PHASE_SEARCHING     = 2    # thermal active, loiter
    centre not yet set
uint8 SOARING_PHASE_THERMAL       = 3    # thermal loiter, centre
    established
```

The field `dds_inhibit_until_us` is set when the operator issues `soar:off`: the FMU ignores DDS attempts to re-enable soaring until that timestamp (5s after the command). The companion can subscribe to this topic and align its publishes (e.g. send `SOARING_OFF` or stop asserting soaring on) before the inhibit window ends. While the inhibit is active, `fw_pos_control` republishes status at 1Hz for packet-loss robustness.

The field `fmu_soaring_phase` reflects what the FMU is *actually executing* each cycle, regardless of what the companion requested. This gives the companion an independent confirmation of the real flight phase, and the value is recorded in ULog via uORB so the phase history is available post-flight. During active soaring, a `SOURCE_PERIODIC` message is emitted at 2Hz so the companion always has a fresh phase estimate.

3.6 Staleness Watchdog

A companion-silence detector is implemented in `FixedwingPositionControl::Run()`:

$$\Delta t_{\text{silence}} = t_{\text{now}} - t_{\text{last_recv}} \quad (1)$$

If $\Delta t_{\text{silence}} > 2\text{s}$ and `soaring_mode` $\neq 0$, the FMU autonomously sets `soaring_mode` $\leftarrow 0$ (off) and disables gliding in TECS. It also publishes `AutosoaringStatus` with `SOURCE_STALENESS_WATCHDOG` and `dds_inhibit_until_us = 0`. This prevents the aircraft from remaining in engine-off glide indefinitely if the companion crashes or the DDS link is lost.

3.7 CLI Override Path

For ground testing without a companion, the Commander module accepts CLI commands:

```
commander mode soar:glide      # mode 1
commander mode soar:polar      # mode 2
commander mode soar:thermal    # mode 3
commander mode soar:bank       # mode 4
commander mode soar:off        # mode 0
```

Each command sends `VEHICLE_CMD_CUSTOM_0` with `param1` equal to the soaring mode integer.

When **enabling** soaring (`param1` $\neq 0$), the CLI path sets `_soaring_local_override = true`, which bypasses:

- altitude floor check (`FW_ALT_MIN`);
- staleness watchdog (no companion expected);
- any active DDS inhibit (the inhibit timer is cleared).

When **disabling** soaring (`soar:off`, `param1 = 0`), `_soaring_local_override` is cleared, a 5s DDS inhibit is started, and the FMU publishes `AutosoaringStatus` with `SOURCE_CLI_SOARING_OFF` and `dds_inhibit_until_us` set to the absolute deadline (see above).

4 Summary of Changes

Aspect	Original	Modified
Mode encoding	Two separate booleans + hidden parameter	Single <code>uint8</code> <code>soaring_mode</code>
Polar selection	PX4 parameter <code>FW_GLIDE_MODE</code>	Encoded in <code>soaring_mode = 2</code>
DDS registration	Not present	<code>/fmu/in/autosoaring_control</code> in <code>dds_topics.yaml</code>
FMU status to companion	Not present	<code>/fmu/out/autosoaring_status</code> + <code>AutosoaringStatus.msg</code>
Turn direction	Not supported	<code>loiter_clockwise</code> bool
Staleness guard	Not present	2-second watchdog in <code>Run()</code> (+ status publish)
CLI commands	<code>soar:glide</code> , <code>soar:thermal</code> , <code>soar:off</code>	+ <code>soar:polar</code> , <code>soar:bank</code>